

Document number: P2643R0

Date: 2022-09-15

Reply to: Gonzalo Brito Gadeschi <gonzalob_at_nvidia.com (<http://nvidia.com>)>

Authors: Gonzalo Brito Gadeschi, Olivier Giroux, Thomas Rodgers

Audience: Concurrency

Improving C++ concurrency features

Revisions

This is the initial revision.

Introduction

When we applied P1135R6 to C++20, we introduced several new concurrency constructs to the C++ concurrency library:

- In `<atomic>`, the member functions `wait`, `notify_one` and `notify_all` were added to class template `atomic<>` and class `atomic_flag`, and free function versions of the same also.
- In `<semaphore>`, the class template `counting_semaphore<>` and class `binary_semaphore` were introduced.
- In `<barrier>` and `<latch>`, the class template `barrier<>` and class `latch` were introduced.

Though each element included was long coming, and had much implementation experience behind it, fresh user feedback tells us that some improvements could still be made.

Proposed direction

The following is a grossly priority-ordered list of requests that users and implementers both have voiced over the last year:

1. Add timed versions of `atomic::wait`.

The primary purpose of this facility is to make it easier to implement other concurrency facilities, but often these other facilities expose timed waiting facilities themselves. Without timed versions of `wait`, the programmer is left to ad-hoc solutions for timed waiting facilities, and perhaps even all waiting facilities. Anecdotally, at least two implementations of C++20 have added internal timed versions of this facility to implement `<semaphore>`.

Adding timed versions of `atomic::wait` removes hurdles to adoption of this facility for its intended purpose.

Adding timed versions of `atomic::wait` will require a discussion of what facilities from `<chrono>` need to be present in `<atomic>` for freestanding implementations.

2. Return the last observed value from `atomic::wait`.

After the return from `wait`, it is common for programs to reload the value of the atomic object. By necessity, the implementation of `wait` already loaded this value, to compare it with the operand supplied and return non-spuriously. This is duplicate work which, in principle, could be optimized away by the compiler but conservatively isn't.

Returning the value from `atomic::wait` is a straightforward way to recover performance lost from the duplicate work.

3. Avoid spurious polling in `atomic::wait` with ***at least one of***:

a. Add an overload of `wait` taking a predicate instead of a value.

When the program is waiting for a condition different from “not equal to”, there is an added re-try loop around the `wait` operation in the program. This loop causes *each* call to `wait` to be performed as if it were the *first* call to `wait`, oblivious to the fact that the program has already been waiting for some time. This leads to re-executing the short-term polling strategy.

Taking a predicate instead of a value allows us to push the program-defined condition inside of `atomic::wait`, delete the outer loop, and allows the implementation to track time spent.

At least two implementations currently implement `atomic::wait` in terms of a `wait` taking a predicate.

b. Add a hint operand to `wait` to steer the internal strategy.

By default, that short-term strategy inside of `wait` is to poll the atomic object's value for some time, so as to avoid limiting the responsiveness of the program to that of the operating system kernel's scheduler. Sometimes, however, it is known that either (a) an event cannot or is not hoped to occur in this short of a window of time, or (b) the program has already supplied its own polling strategy before the call to `wait`, or © this call to `wait` is not the first and should be considered a long-term wait.

Taking a hint would let the program indicate whether the short-term strategy of `atomic::wait` should execute or not.

4. Add timed versions of `barrier::wait` and `latch::wait` also.

Since every waiting facility in the concurrency library has timed wait functions at this point, it makes sense to add timed versions of these as well.

Although this is a very weak reason to do anything, there is also no clear reason why we should not do it.

Design

The design of the features above is mostly orthogonal, and this section explores them independently.

1. **Return last observed value from `atomic::wait` APIs:** solved as `void T wait(...)`;

2. **Fallible and timed versions of wait APIs:**

- Solved by adding:

- `optional<T> try_wait(...)`,
- `optional<T> try_wait_for(..., chrono::duration<Rep, Period> const&)`, and
- `optional<T> try_wait_until(..., chrono::time_point<Clock, Duration> const&)`

methods that return `nullopt` if the wait operation did not synchronize, and an `optional<T>` containing the `T` value observed if it did synchronize.

Wording

Return last observed value from `atomic::wait`

To [atomics.ref.generic.general] (<http://eel.is/c++draft/atomics.ref.generic.general>):

```
namespace std {
    template<class T> struct atomic_ref { // [atomics.ref.generic.general]
        void wait(T, memory_order = memory_order::seq_cst) const noexcept;
    };
}
```

UNRESOLVED QUESTION: all `atomic_ref` types are missing `volatile` wait overloads?

To [atomics.ref.ops] (http://eel.is/c++draft/atomics#lib:wait,atomic_ref%3cT%3e):

```
void wait(T old, memory_order order = memory_order::seq_cst) const noexcept;
```

- *Preconditions:* `order` is neither `memory_order::release` nor `memory_order::acq_rel`.
- *Effects:* Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares its value representation for equality against that of `old`.
 2. If they compare unequal, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks:* This function is an atomic waiting operation (`atomics.wait` (<http://eel.is/c++draft/atomics#wait>)) on atomic object `*ptr`.

To [atomics.ref.int] (<http://eel.is/c++draft/atomics.ref.int>):

```
namespace std {
    template<> struct atomic_ref<integral> {
        void wait(integral, memory_order = memory_order::seq_cst) const noexcept;
    };
}
```

To [atomics.ref.float] (<http://eel.is/c++draft/atomics.ref.float>):

```
namespace std {
    template<> struct atomic_ref<floating-point> {
        void wait(floating-point, memory_order = memory_order::seq_cst) const noexcept;
    };
}
```

To [atomics.ref.pointer] (<http://eel.is/c++draft/atomics.ref.pointer>):

```

namespace std {
    template<class T> struct atomic_ref<T*> {
        voidT* wait(T*, memory_order = memory_order::seq_cst) const noexcept;
    };
}

```

To [atomics.types.generic.general] (<http://eel.is/c++draft/atomics.types.generic.general>):

```

namespace std {
    template<class T> struct atomic {
        voidT wait(T, memory_order = memory_order::seq_cst) const volatile noexcept;
        voidT wait(T, memory_order = memory_order::seq_cst) const noexcept;
    };
}

```

To [atomics.types.operations] (<http://eel.is/c++draft/atomics.types.operations#lib:wait,atomic>):

```

voidT wait(T old, memory_order order = memory_order::seq_cst) const volatile noexc
voidT wait(T old, memory_order order = memory_order::seq_cst) const noexcept;

```

- *Preconditions*: order is neither `memory_order::release` nor `memory_order::acq_rel`.
- *Effects*: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares its value representation for equality against that of `old`.
 2. If they compare unequal, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks*: This function is an atomic waiting operation (`atomics.wait` (<http://eel.is/c++draft/atomics#wait>)).

To [atomics.types.int] (<http://eel.is/c++draft/atomics.types.int>):

```

namespace std {
    template<> struct atomic<integral> {
        voidT wait(integral, memory_order = memory_order::seq_cst) const volatile noex
        voidT wait(integral, memory_order = memory_order::seq_cst) const noexcept;
    };
}

```

To `[atomics.types.float]` (<http://eel.is/c++draft/atomics.types.float>):

```
namespace std {
    template<> struct atomic<floating-point> {
        voidT wait(floating-point, memory_order = memory_order::seq_cst) const volatile
        voidT wait(floating-point, memory_order = memory_order::seq_cst) const noexcept
    };
}
```

To `[atomics.types.pointer]` (<http://eel.is/c++draft/atomics.types.pointer>):

```
namespace std {
    template<class T> struct atomic<T*> {
        voidT* wait(T*, memory_order = memory_order::seq_cst) const volatile noexcept;
        voidT* wait(T*, memory_order = memory_order::seq_cst) const noexcept;
    };
}
```

To `[util.smartptr.atomic.shared]` (<http://eel.is/c++draft/util.smartptr.atomic.shared>):

```
namespace std {
    template<class T> struct atomic<shared_ptr<T>> {
        voidshared_ptr<T> wait(shared_ptr<T> old, memory_order = memory_order::seq_cst
    };
}
```

and

```
voidshared_ptr<T> wait(shared_ptr<T> old, memory_order order = memory_order::seq_c
```

- *Preconditions*: `order` is neither `memory_order::release` nor `memory_order::acq_rel`.
- *Effects*: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares it to `old`.
 2. If the two are not equivalent, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks*: Two `shared_ptr` objects are equivalent if they store the same pointer and either share ownership or are both empty. This function is an atomic waiting operation

(`atomics.wait` (<http://eel.is/c++draft/atomics.wait>)).

To [`util.smartptr.atomic.weak`] (<http://eel.is/c++draft/util.smartptr.atomic.weak>):

```
namespace std {
    template<class T> struct atomic<weak_ptr<T>> {
        voidweak_ptr<T> wait(weak_ptr<T> old, memory_order = memory_order::seq_cst) co
    };
}
```

```
voidweak_ptr<T> wait(weak_ptr<T> old, memory_order order = memory_order::seq_cst)
```

- *Preconditions*: order is neither `memory_order::release` nor `memory_order::acq_rel`.
- *Effects*: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares it to `old`.
 2. If the two are not equivalent, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.
- *Remarks*: Two `weak_ptr` objects are equivalent if they store the same pointer and either share ownership or are both empty. This function is an atomic waiting operation (`atomics.wait` (<http://eel.is/c++draft/atomics#wait>)).

No changes to [`atomics.nonmembers`] (<http://eel.is/c++draft/atomics.nonmembers>) are needed.

No changes to [`atomic.flag`] (<http://eel.is/c++draft/atomic.flag>)'s `wait` APIs are needed.

Fallible and timed-versions of `::wait` APIs

To [`atomics.ref.generic.general`] (<http://eel.is/c++draft/atomics.ref.generic.general>):

```

namespace std {
    template<class T> struct atomic_ref { // [atomics.ref.generic.general]

        optional<T> try wait(T, memory_order = memory_order::seq_cst) const noexcept;
        template <class Rep, class Period>
        optional<T> try wait for(
            T, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<T> try wait until(
            T, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
    };
}

```

UNRESOLVED QUESTION: all atomic ref types are missing volatile wait overloads?

To [atomics.ref.ops] (http://eel.is/c++draft/atomics#lib:wait,atomic_ref%3cT%3e):

```

optional<T> try wait(T old, memory_order order = memory_order::seq_cst) const noex

```

- Preconditions: order is neither memory_order::release nor memory_order::acq_rel.
- Effects: Performs the following steps in order:
 1. Evaluates load(order) and compares its value representation for equality against that of old.
 2. If they compare unequal, returns the result of the evaluation of load(order) in the previous step.
 3. Otherwise, there is no effect and it returns nullopt.

Remarks: This function is an atomic waiting operation (atomics.wait (<http://eel.is/c++draft/atomics.wait>)).

```

template <class Rep, class Period>
optional<T> try_wait_for(T old,
    chrono::duration<Rep, Period> const& rel_time,
    memory_order order = memory_order::seq_cst
) const noexcept;
template <class Clock, class Duration>
optional<T> try_wait_until(T old,
    chrono::time_point<Clock, Duration> const& abs_time,
    memory_order order = memory_order::seq_cst
) const noexcept;

```

- Preconditions: order is neither `memory_order::release` nor `memory_order::acq_rel`.
- Effects: Repeatedly performs the following steps, in order:
 1. Evaluates `load(order)` and compares its value representation for equality against that of `old`.
 2. If they compare unequal, returns the result of the evaluation of `load(order)` in the previous step.
 3. Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously or the timeout expired. If it is unblocked by the timeout there is no effect and it returns `nullopt`.

The timeout expires (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)) when the current time is after `abs_time` (for `try_wait_until`) or when at least `rel_time` has passed from the start of the function (for `try_wait_for`).

An implementation should ensure that `try_wait_for` and `try_wait_until` do not consistently return `nullopt` in the absence of contending atomic operations.

- Throws: Timeout-related exceptions (`thread.req.timing` (<http://eel.is/c++draft/thread.req.timing>)).
- Remarks: This function is an atomic waiting operation (`atomics.wait` (<http://eel.is/c++draft/atomics#wait>)) on atomic object `*ptr`.

To `[atomics.ref.int]` (<http://eel.is/c++draft/atomics.ref.int>):

```

namespace std {
    template<> struct atomic_ref<integral> {
        optional<integral> try wait(integral, memory_order = memory_order::seq_cst) co
        template <class Rep, class Period>
        optional<integral> try wait for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<integral> try wait until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
    };
}

```

To [atomics.ref.float] (<http://eel.is/c++draft/atomics.ref.float>):

```

namespace std {
    template<> struct atomic_ref<floating-point> {
        optional<floating-point> try wait(floating-point, memory_order = memory_order:
        template <class Rep, class Period>
        optional<floating-point> try wait for(
            floating-point, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<floating-point> try wait until(
            floating-point, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
    };
}

```

To [atomics.ref.pointer] (<http://eel.is/c++draft/atomics.ref.pointer>):

```

namespace std {
    template<class T> struct atomic_ref<T*> {
        optional<T*> try_wait(T*, memory_order = memory_order::seq_cst) const noexcept
        template <class Rep, class Period>
        optional<T*> try_wait for(
            T*, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<T*> try_wait until(
            T*, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
    };
}

```

To [atomics.types.generic.general] (<http://eel.is/c++draft/atomics.types.generic.general>):

```

namespace std {
    template<class T> struct atomic {
        optional<T> try_wait(T, memory_order = memory_order::seq_cst) const noexcept;
        optional<T> try_wait(T, memory_order = memory_order::seq_cst) const volatile n
        template <class Rep, class Period>
        optional<T> try_wait for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        optional<T> try_wait for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
        template <class Clock, class Duration>
        optional<T> try_wait until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<T> try_wait until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
    };
}

```

To [atomics.types.operations] (<http://eel.is/c++draft/atomics.types.operations#lib:wait,atomic>):

```
optional<T> try_wait(T, memory_order = memory_order::seq_cst) const noexcept;  
optional<T> try_wait(T, memory_order = memory_order::seq_cst) const volatile noexc  
template <class Rep, class Period>  
optional<T> try_wait_for(T old,  
                        chrono::duration<Rep, Period> const& rel time,  
                        memory_order order = memory_order::seq_cst  
                        ) const noexcept;  
template <class Rep, class Period>  
optional<T> try_wait_for(T old,  
                        chrono::duration<Rep, Period> const& rel time,  
                        memory_order order = memory_order::seq_cst  
                        ) const volatile noexcept;  
template <class Clock, class Duration>  
optional<T> try_wait_until(T old,  
                        chrono::time_point<Clock, Duration> const& abs time,  
                        memory_order order = memory_order::seq_cst  
                        ) const noexcept;  
template <class Clock, class Duration>  
optional<T> try_wait_until(T old,  
                        chrono::time_point<Clock, Duration> const& abs time,  
                        memory_order order = memory_order::seq_cst  
                        ) const volatile noexcept;
```

EDITORIAL: analogous to `atomic_ref`. Intentionally left out from the current revision of this paper.

To [atomics.types.int] (<http://eel.is/c++draft/atomics.types.int>):

```

namespace std {
    template<> struct atomic<integral> {
        optional<integral> try wait(integral, memory_order = memory_order::seq_cst) co
        optional<integral> try wait(integral, memory_order = memory_order::seq_cst) co
        template <class Rep, class Period>
        optional<integral> try wait for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Rep, class Period>
        optional<integral> try wait for(
            integral, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
        template <class Clock, class Duration>
        optional<integral> try wait until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<integral> try wait until(
            integral, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
    };
}

```

To [atomics.types.float] (<http://eel.is/c++draft/atomics.types.float>):

```

namespace std {
    template<> struct atomic<floating-point> {
        optional<floating-point> try wait(floating-point, memory_order = memory_order::
optional<floating-point> try wait(floating-point, memory_order = memory_order:
        template <class Rep, class Period>
        optional<floating-point> try wait for(
            floating-point, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Rep, class Period>
        optional<floating-point> try wait for(
            floating-point, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
        template <class Clock, class Duration>
        optional<floating-point> try wait until(
            floating-point, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<floating-point> try wait until(
            floating-point, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
    };
}

```

To [atomics.types.pointer] (<http://eel.is/c++draft/atomics.types.pointer>):

```

namespace std {
    template<class T> struct atomic<T*> {
        optional<T*> try_wait(T*, memory_order = memory_order::seq_cst) const noexcept
        optional<T*> try_wait(T*, memory_order = memory_order::seq_cst) const volatile
        template <class Rep, class Period>
        optional<T*> try_wait_for(
            T*, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Rep, class Period>
        optional<T*> try_wait_for(
            T*, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
        template <class Clock, class Duration>
        optional<T*> try_wait_until(
            T*, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<T*> try_wait_until(
            T*, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
    };
}

```

To [util.smartptr.atomic.shared] (<http://eel.is/c++draft/util.smartptr.atomic.shared>):

```

namespace std {
    template<class T> struct atomic<shared_ptr<T>> {
        optional<shared_ptr<T>> try_wait(shared_ptr<T>, memory_order = memory_order::s
        template <class Rep, class Period>
        optional<shared_ptr<T>> try_wait_for(
            shared_ptr<T>, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<shared_ptr<T>> try_wait_until(
            shared_ptr<T>, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
    };
}

```

EDITORIAL: analogous to the `try_wait` APIS of `atomic_ref`, with `shared_ptr / weak_ptr` tweaks. Intentionally left out of the current revision of this paper.

To `[util.smartptr.atomic.weak]` (<http://eel.is/c++draft/util.smartptr.atomic.weak>):

```
namespace std {
    template<class T> struct atomic<weak_ptr<T>> {
        optional<weak_ptr<T>> try_wait(weak_ptr<T>, memory_order = memory_order::seq_c
        template <class Rep, class Period>
        optional<weak_ptr<T>> try_wait_for(
            weak_ptr<T>l, chrono::duration<Rep, Period> const& rel time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        optional<weak_ptr<T>> try_wait_until(
            weak_ptr<T>, chrono::time_point<Clock, Duration> const& abs time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
    };
}
```

EDITORIAL: analogous to the `try_wait` APIS of `atomic_ref`, with `shared_ptr / weak_ptr` tweaks. Intentionally left out of the current revision of this paper.

EDITORIAL: No changes to `[atomics.nonmembers]` (<http://eel.is/c++draft/atomics.nonmembers>) are needed.

To `[atomic.flag]` (<http://eel.is/c++draft/atomics.flag>):

```

namespace std {
    struct atomic_flag {
        bool try_wait(bool, memory_order = memory_order::seq_cst) const noexcept;
        bool try_wait(bool, memory_order = memory_order::seq_cst) const volatile noexcept;
        template <class Rep, class Period>
        bool try_wait_for(
            bool, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Rep, class Period>
        bool try_wait_for(
            bool, chrono::duration<Rep, Period> const& rel_time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
        template <class Clock, class Duration>
        bool try_wait_until(
            bool, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const noexcept;
        template <class Clock, class Duration>
        bool try_wait_until(
            bool, chrono::time_point<Clock, Duration> const& abs_time,
            memory_order = memory_order::seq_cst
        ) const volatile noexcept;
    };
}

```

```

bool atomic_flag::try_wait(const atomic_flag* object, bool old) noexcept;
bool atomic_flag::try_wait(const volatile atomic_flag* object, bool old) noexcept;
bool atomic_flag::try_wait_explicit(const atomic_flag* object, bool old, memory_order order) const;
bool atomic_flag::try_wait_explicit(const volatile atomic_flag* object, bool old, memory_order order) const volatile;
bool atomic_flag::try_wait(bool old, memory_order order = memory_order::seq_cst) const;
bool atomic_flag::try_wait(bool old, memory_order order = memory_order::seq_cst) const volatile;

```

For `atomic_flag::try_wait`, let `order` be `memory_order::seq_cst`. Let `flag` be `object` for the non-member functions, and `this` for the member functions.

- **Preconditions**: `order` is neither `memory_order::release` nor `memory_order::acq_rel`.
- **Effects**: Repeatedly performs the following steps, in order:
 1. Evaluates `flag->test(order) != old`.
 2. If the result of that evaluation is `true`, returns `true`.
 3. Otherwise, it has no effects and returns `false`.

- *Remarks:* This function is an atomic waiting operation (`atomics.wait` (<http://eel.is/c++draft/atomics.wait>)).

EDITORIAL: analogous for the `atomic_flag_try_wait_for` / `_until` APIs. Intentionally omitted from the current revision of this paper.

UNRESOLVED QUESTION: do we need to change something else for the non-member versions of `try_wait`, `try_wait_for`, and `try_wait_until` operations?

UNRESOLVED QUESTION: do we need to define a “try-wait” atomic operation in `atomics.wait` (<http://eel.is/c++draft/atomics.wait>)?

To [`thread.barrier` (<http://eel.is/c++draft/thread.barrier>)]:

```
namespace std {
    template <class Completion Function>
    class barrier {

    public:
        bool try_wait(arrival token&& tok) const;
        template <class Rep, class Period>
        bool try_wait_for(arrival token&& tok, chrono::duration<Rep, Period> const& re
        template <class Clock, class Duration>
        bool try_wait_until(arrival token&& tok, chrono::time_point<Clock, Duration> c
    };
}
```

UNRESOLVED QUESTION: should we remove `const` qualification from the new APIs if P2588 (<http://wg21.link/p2588>) is accepted?

EDITORIAL: these changes are compatible with both adding `try_wait` overloads that accept a `memory_order` (P2628 (<http://wg21.link/p2628>)) and `try_wait` overloads that accept a `bool parity` instead of an `arrival_token` (P2629 (<http://wg21.link/p2629>)).

```
bool try_wait(arrival token&& arrival) const;
```

- *Preconditions:* `arrival` is associated with the phase synchronization point for the current phase or the immediately preceding phase of the same barrier object.
- *Effects:* If `arrival` is associated with the synchronization point for a previous phase, the call returns `true` immediately without blocking. Otherwise, there are no effects, and the call returns `false`.

- Throws: `system_error` when an exception is required (`thread.req.exception` (<http://eel.is/c++draft/thread.req.exception>)).
- Error conditions: Any of the error conditions allowed for mutex types (`thread.mutex.requirements.mutex` (<http://eel.is/c++draft/thread.mutex.requirements.mutex>)).

UNRESOLVED QUESTION: if P2588 (<http://wg21.link/p2588>) is accepted, then `try_wait` is able to complete the phase and the *Effects* clause needs updating, e.g., as follows: “[...] Otherwise, if all threads have arrived `try_wait` may complete the phase and return `true`, or the call has no effects and returns `false`.”.

```
template <class Rep, class Period>
bool try_wait_for(arrival token&& tok, chrono::duration<Rep, Period> const& rel ti
template <class Clock, class Duration>
bool try_wait_until(arrival token&& tok, chrono::time_point<Clock, Duration> const
```

EDITORIAL: `try_wait_for` and `try_wait_until` shall have analogous semantics.

To `thread.latch` (<http://eel.is/c++draft/thread.latch>):

```
namespace std {
    class latch {
    public:
        template <class Rep, class Period>
        bool try_wait_for(chrono::duration<Rep, Period> const& rel time) const;
        template <class Clock, class Duration>
        bool try_wait_until(chrono::time_point<Clock, Duration> const& abs time) const
    };
}
```

```
template <class Rep, class Period>
bool try_wait_for(chrono::duration<Rep, Period> const& rel time) const;
template <class Clock, class Duration>
bool try_wait_until(chrono::time_point<Clock, Duration> const& abs time) const;
```

EDITORIAL: semantics intentionally omitted from the current revision of this paper.